

PRÄDIKATE

Prädikate sind Aussagen über mathematische Objekte, die wahr oder falsch sein können. «Funktionale» Prädikate mit Beispielen Rückgabewerten: $P, Q(n), R(x, y, z)$.

Regulär

DEA

NEA

Regex

Kontextfrei

CFG

PDA

Palindrome

$\{0^n1^n \mid n \geq 0\}$

Turing-erkennbar

Primzahlen

$\{a^n b^n c^n \mid n \geq 0\}$

Alle Sprachen

Überabzählbar

Überabzählbar

Begriff

Bedeutung

Abtrennungsregel

$(A \wedge (A \Rightarrow B)) \Rightarrow B$

Kommutativität

$(A \wedge B) \Leftrightarrow (B \wedge A)$
 $(A \vee B) \Leftrightarrow (B \vee A)$

Assoziativität

$A \wedge (B \wedge C) \Leftrightarrow (A \wedge B) \wedge C$
 $A \vee (B \vee C) \Leftrightarrow (A \vee B) \vee C$

Distributivität

$A \wedge (B \vee C) \Leftrightarrow (A \wedge B) \vee (A \wedge C)$
 $A \vee (B \wedge C) \Leftrightarrow (A \vee B) \wedge (A \vee C)$

Absorption

$A \vee (A \wedge B) \Leftrightarrow A$
 $A \wedge (A \vee B) \Leftrightarrow A$

Idempotenz

$A \vee A = A$
 $A \wedge A = A$

Doppelte Negation

$\neg(\neg A) \Leftrightarrow \neg\neg A \Leftrightarrow A$

Konstanten

$W = \text{wahr}$
 $F = \text{falsch}$

de Morgan

$\neg(A \wedge B) \Leftrightarrow \neg A \vee \neg B$
 $\neg(A \vee B) \Leftrightarrow \neg A \wedge \neg B$

NORMALFORMEN

Normalformen (allgemein, **kanonische Formen**) helfen, das Vergleichsproblem zu lösen (ob zwei Aussagen dieselben sind).

$$P_1 \wedge \dots \wedge P_n = \forall i \in \{1, \dots, n\} (P_i) = \bigwedge_{i=1}^n P_i$$
$$P_1 \vee \dots \vee P_n = \exists i \in \{1, \dots, n\} (P_i) = \bigvee_{i=1}^n P_i$$

NEGATION

Nicht für alle = Es gibt einen Fall, für den nicht

$\neg \forall i \in \{1, \dots, n\} (P_i) \Leftrightarrow \neg(P_1 \wedge \dots \wedge P_n) \Leftrightarrow \neg P_1 \vee \dots \vee \neg P_n \Leftrightarrow \exists i \in \{1, \dots, n\} \neg P_i$

MENGEN

KONSTRUKTION

Grundmenge G , Prädikat $P(x)$. Konstruierte Menge $A = \{x \in G \mid P(x)\}$

OPERATIONEN

Begriff

Bedeutung

Vereinigung

$A \cup B = \{x \mid x \in A \vee x \in B\}$

Schnittmenge

$A \cap B = \{x \mid x \in A \wedge x \in B\}$

Komplement

$\bar{A} = \{x \mid x \notin A\}$

Differenz

$A \setminus B = \{x \in A \mid x \notin B\}$

TUPEL

$A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$

n -Tupel:
$$\prod_{i=1}^n A_i = \{(a_1, a_2, \dots, a_n) \mid a_i \in A_i\}$$

BEWEISE

Eine Folge von logischen Schlüssen, die zeigen, dass die Aussage aus den gegebenen Voraussetzungen folgt.

Beweistechnik

Anwendungsfall

Konstruktiver Beweis

Liefert explizite «Lösung» / Algorithmus

Widerspruchsbeweis

Geeignet für Unmöglichkeitss Aussagen. Z.B. $\sqrt{2} \notin \mathbb{Q}$

Vollständige Induktion

Für Aussagen, die für alle $n \in \mathbb{N}$ gelten.

SPRACHEN

$A = (Q, \Sigma, \delta, q_0, F)$

Zustände:

$Q = \{q_0, q_1, \dots, q_n\}$

Übergangsfunktion:

$\delta: Q \times \Sigma \rightarrow Q$

Startzustand:

$q_0 \in Q$

Akzeptierzustände:

$F \subseteq Q$

Wichtig: Ein Pfeil für jedes Zeichen in jedem Zustand für validen DEA

WÖRTER EINER REGULÄREN SPRACHE

ÜBERGANGSFUNKTIONEN FÜR WÖRTER

$\delta: Q \times \Sigma^* \rightarrow Q: (q, w = a_1, \dots, a_n) \mapsto \delta(\dots \delta(q, a_1), a_2), \dots, a_n)$

Übergänge

ausgehend von q für Zeichen $a_1, \dots, a_n$ nacheinander anwenden.

WORT AKZEPTIEREN

Der DEA $A = (\Sigma, Q, q_0, \delta, F)$ **akzeptiert** das Wort $w \in \Sigma^*$, wenn er A von Startzustand in einen Akzeptierzustand $\delta(q_0, w) \in F$ überführt.

AKZEPTIERTE SPRACHE, REGULÄRE SPRACHE

Gegeben ein DEA $A = (\Sigma, Q, q_0, \delta, F)$. Die von A **akzeptierte Sprache** ist

$L(A) = \{w \in \Sigma^* \mid A \text{ akzeptiert } w\} = \{w \in \Sigma^* \mid \delta(q_0, w) \in F\}$

Die Sprache $L \subseteq \Sigma^*$ heisst **regulär**, wenn es einen DEA A gibt mit $L(A) = L$

Beispiel 3: DEA, der die Sprache $L = \{w \in \Sigma^* \mid w \text{ ist eine ungerade Zahl im Dreiersystem}\}$ akzeptiert

MYHILL-NERODE AUTOMAT

Für ein Wort $w \in \Sigma^*$ setze $L(w) = \{w' \in \Sigma^* \mid ww' \in L\}$. Insbesondere $L(\varepsilon) = L$

Gegeben: reguläre Sprache L über Σ . Rekonstruiere A mit:

- $Q = \{L(w) \mid w \in \Sigma^*\}$
- $q_0 = L(\sigma) = L$
- $F = \{L(w) \in Q \mid \varepsilon \in L(w)\}$
- $\delta(L(w), a) = L(wa)$

Gleicher Zustand $\Leftrightarrow$ gleiches $L(w)$

Beispiel 4: $\Sigma = \{0, 1\}, L = \{w \in \Sigma^* \mid |w|_0 \text{ gerade}\}$

MINIMALAUTOMAT

Ziel: Nicht unterscheidbare Zustände zusammenlegen

Vorgehen:

- 1) Akzeptierzustand $\neq$ Nichtakzeptierzustand
- 2) Iteration: alle unterscheidbaren Paare suchen
- 3) Verbleibende Paare sind ununterscheidbar $\rightarrow$ zusammenlegen

Beispiel 5

Algorithmus

- 1) $q \in F, q \notin F$
- 2) Unterscheidbar nach Übergang
- 3) Iteration bis keine Änderung mehr $\Rightarrow q_1$ und q_2 sind nicht unterscheidbar

PUMPING LEMMA

Ist L eine reguläre Sprache, dann gibt es $N \in \mathbb{N}$, die pumping length so, dass jedes Wort $w \in L$ mit $|w| \geq N$ in drei Teile $w = xyz$ zerlegt werden kann mit:

- 1) $|xy| \leq N$
- 2) $|y| > 0$
- 3) $xy^kz \in L \forall k \in \mathbb{N}$

Was zeichnet reguläre Sprachen aus?

- Reguläre Ausdrücke
- Lange Wörter: * kommt im regulären Ausdruck vor
- Blöcke mit * können beliebig oft wiederholt werden $\Rightarrow$ Wörter können «aufgepumpt» werden

BEWEIS

Behauptung: Die Sprache $L \subseteq \Sigma^*$ ist nicht regulär.

Beweis (Widerspruch):

- 1) Annahme: L ist regulär
- 2) Gemäss Pumping Lemma gibt es die Pumping Length N
 - Es darf keine Annahme über die konkrete Grösse von N gemacht werden!
- 3) Wähle ein Wort $w \in L$ mit $|w| \geq N$
 - Die Definition **muss** N verwenden!
- 4) Aufteilung des Wortes gemäss Pumping Lemma $w = xyz, |xy| \leq N, |y| > 0$
- 5) Auswirkung des Pumpens
 - $xy^kz \notin L$ für mindestens ein $k \in \mathbb{N}$ (mit Begründung!)
- 6) Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

Beispiel 6: $L = \{0^n 1^n \mid n \geq 0\}$ ist nicht regulär

- 1) Annahme: L ist regulär
- 2) $\exists N \in \mathbb{N}$, Pumping Length
- 3) $w = 0^N 1^N$
- 4) Unterteilung $w = xyz$

TODO: reduce confusion

5) Pumpen: nur die Anzahl der 1 wird erhöht, Anzahl 0 bleibt

6) $xy^kz \notin L$ für $k \neq 1$, im Widerspruch zum Pumping Lemma

NICHTDETERMINISTISCHE ENDLICHE AUTOMATEN (NEA)

Definition

$A = (Q, \Sigma, \delta, q_0, F)$

- Endliche Menge von Zuständen: Q
- Alphabet: Σ
- Übergangsfunktion: $\delta: Q \times \Sigma \rightarrow P(Q)$
- Startzustand: $q_0 \in Q$
- Akzeptierzustände: $F \subseteq Q$

Akzeptieren

Ein NEA A **akzeptiert** das Wort $w \in \Sigma^*$, wenn es eine **Wahl** von Übergängen gibt, derart, dass das Wort w den Automaten in einen Akzeptierzustand überführt.

Faustregeln

- Nur genau diejenigen Pfeile einzeichnen, die man zum Akzeptieren braucht.
- Erlaube Alternativen

Beispiel 7: $L = \{w \in \Sigma^* \mid w \text{ endet mit zwei 0}\}$

DEA-Lösung

NEA-Lösung

UMGANG MIT MEHRDEUTIGEN ÜBERGÄNGEN

a-Übergang von q_1 aus nicht eindeutig

Welchen Übergang soll man nehmen?

- Beide Möglichkeiten probieren und akzeptieren, falls eine der Möglichkeiten auf einen Akzeptierzustand führt.
- Zufallsgenerator (probabilistischer endlicher autom., PEA)

AutoSpr | FS26

Seite 1

Seite 2

Mit dem Pumping Lemma kann man beweisen, dass eine Sprache **nicht** kontextfrei ist.

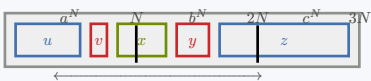
Beispiel 15: $\{a^n b^n c^n \mid n \geq 0\}$

1) Annahme: L kontextfrei

2) Pumping length N

3) Wort: $w = a^N b^N c^N$

4) Zerlegungen:



5) Beim Pumpen nimmt die Anzahl der a und b zu, nicht aber die Anzahl der c
 $\Rightarrow uv^k xy^k z \notin L \forall k \neq 1$

6) Widerspruch: L nicht kontextfrei

Herleitung

Grammatik G in CNF

$$w \in L(G) \Rightarrow S \overset{*}{\Rightarrow} w$$

Wiederverwendete Variable

$|w| \geq N$ gross genug $\Rightarrow$ Variablen werden im Parse Tree wiederverwendet
Die «unterste» wiederverwendete Variable A erzeugt zwei Wörter:

$A \Rightarrow vxy$
 $A \Rightarrow x$

Pumpen

$A \overset{*}{\Rightarrow} vxy$ anstelle des «untersten» $A \overset{*}{\Rightarrow} x$ verwenden

TODO:
diagram

UNENDLICH

Begriff	Bedeutung
Endlich	Eine Menge A heisst endlich , wenn jede Injektion $A \rightarrow A$ auch eine Bijektion ist
Abzählbar unendlich	Eine Menge A heisst abzählbar unendlich , wenn es eine Bijektion $\mathbb{N} \rightarrow A$ gibt, also $A \approx \mathbb{N}$. Bsp: $\mathbb{R}, \mathbb{Z}, \mathbb{Q}, \Sigma^*$, Menge DEAs/NEAs/PDAs/CFGs
Überabzählbar unendlich	Eine Menge A heisst überabzählbar unendlich , wenn sie nicht abzählbar unendlich ist. Bsp: $\mathbb{C}, P(\mathbb{N})$, Menge aller Sprachen $P(\Sigma^*)$
Gleich mächtig	Mengen A und B heissen gleich mächtig , $A \approx B$, wenn es eine Bijektion $A \rightarrow B$ gibt.
Unendlich	Eine Menge A heisst unendlich , wenn sie gleich mächtig wie eine Teilmenge ist

A, B, A_k abzählbar unendlich, $k \in \mathbb{N}$:

- 1) $A \cup B, \bigcup_k A_k$ abzählbar unendlich
- 2) $A \times B$ abzählbar unendlich
- 3) Abzählbar unendlich: $\mathbb{N}, \mathbb{Z}, \mathbb{Q}$
- 4) $P(A)$ überabzählbar unendlich
- 5) $\mathbb{R}$ überabzählbar unendlich

TURING-MASCHINE (TM)

Merhere Stacks (Speicherzellen) = RAM (Band).

Jeder DEA oder NEA kann damit emuliert werden.

- 1) Der Speicher ist unbegrenzt
- 2) In jeder Zelle genau ein Zeichen
- 3) Es ist immer nur eine Speicherzelle einsehbar
- 4) Der Inhalt der aktuellen Zelle kann beliebig verändert werden
- 5) Bewegung immer nur eine Zelle nach links oder rechts
- 6) Kein weiterer Speicher (nur die Zustände eines endlichen Automaten und das eine Zeichen)

Begriff	Bedeutung
Record	Speichereinheit fester grösse
Bandalphabet	Alphabet Γ , welches aus Speicherzellen (Stacks) besteht
Schreib-/Lesekopf	Aktuelle Schreib-/Leseposition

Definition

(deterministische) Turing-Maschine

$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$

Q : Zustände

Σ : Alphabet

Γ : Bandalphabet, $\cup \in \Gamma \setminus \Sigma$

δ : $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$

$q_0 \in Q$: Startzustand

$q_{\text{accept}} \in Q$: Akzeptierzustand

$q_{\text{reject}} \in Q$: Ablehnzustand, $q_{\text{accept}} \neq q_{\text{reject}}$

ARBEITSWEISE

Programmablauf

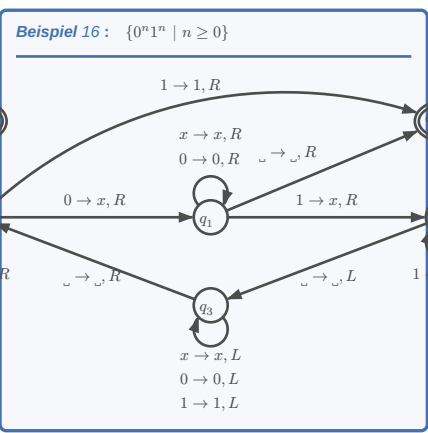
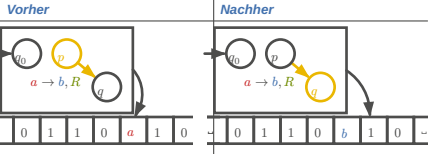
– Input-Wort w auf Band

– Schreib- / Lesekopf positioniert auf 1. Zeichen

– Maschine starten, $t(w)$ Einzelschritte ausführen

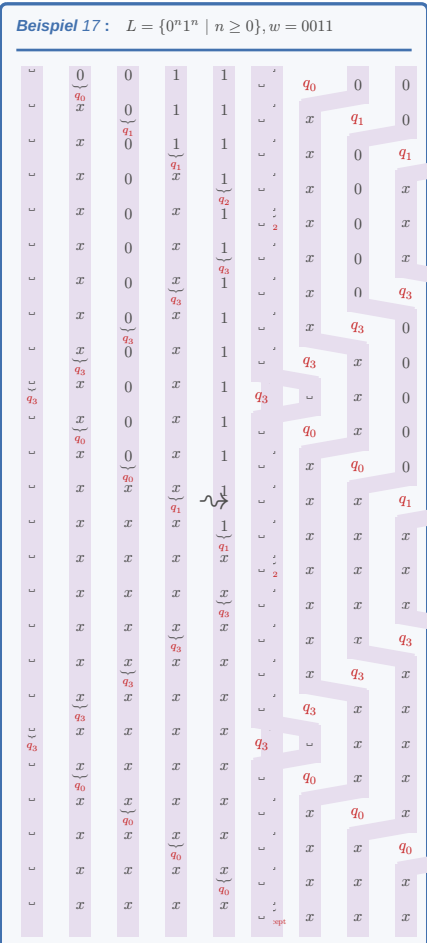
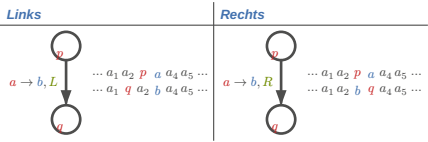
– Maschine hält in q_{accept} oder q_{reject} : Wort w akzeptiert / Verworfen

Laufzeit: $t(w), t(n) = \max\{t(w) \mid |w| \leq n\}$



PROTOKOLLIERUNG

Der Gang der Berechnung mit einer TM kann dokumentiert werden, indem der Bandinhalt nach jedem einzelnen Verarbeitungsschritt protokolliert wird.



VARIANTEN

Laufzeit: Die Laufzeit einer TM M bei der Verarbeitung eines Wortes $w \in \Sigma^*$ ist die Anzahl $t(w)$ der Schritte, die die Turing-Maschine ausführt, bis sie anhält.

NICHTDETERMINISTISCHE TM

Übergangsfunktion

Bei jedem Übergang maximal N verschiedene Möglichkeiten: $\delta: Q \times \Gamma \rightarrow P(Q \times \Gamma \times \{L, R\})$
 $\Rightarrow$ Mehrere mögliche Berechnungswege

Wort akzeptieren

$w \in L(M)$ wenn es einen Berechnungsweg gibt, der zu q_{accept} führt. (auch wenn es Wege gibt, die auf q_{reject} führen!)

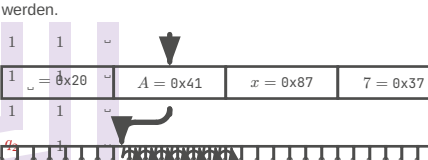
Simulationsidee

Alle Berechnungswege durchführen, maximal $N^{t(n)}$
Simulierbar in $O(N^{t(n)}) = 2^{O(t(n))}$

- Simulation auf Standard-TM**
- Verwende 3 Bänder:
- 1) Arbeitsband
- 2) Kopie von w
- 3) Liste aller Folgen von Wahlmöglichkeiten
- Simulation
- 1) Kopiere w von Band 2 auf Band 1
- 2) Führe TM aus auf Band 1 mit Wahlmöglichkeiten von Band 3
- 3) Inkrementiere zur nächsten Folge von Wahlmöglichkeiten auf Band 3

VERSCHIEDENE BANDALPHABETE

Jedes andere Alphabet als $\Sigma = \{0, 1\}$ kann binär codiert werden.



Simulierbar in Zeit $O(t(n))$

MEHRSPURMASCHINE

TODO:
buch ...250... slides 14

Simulierbar in Zeit $O(t(n))$

MEHRBANDMASCHINE

Um eine Mehrbandmaschine mit n Bändern auf einer einfacheren Maschine zu simulieren, kann die unabhängige Bewegung der Schreib-/Leseköpfe mithilfe eines $2n$ -spurigen Bandes (Daten+ Zeiger für Schreib-/Lesekopf) nachgebildet werden.

TODO:
buch ...250... slides 14

Simulierbar in Zeit $O(t(n)^2)$

Harvard- und von Neumann-Architektur

Die AVR-Mikrokontroller-Familie von Atmel verwendet intern drei unabhängige Datenpfade:

– Flash-Speicher, der den Programmcode enthält

– Statisches RAM, welches zur Laufzeit veränderliche Daten speichert

– Peripheriegeräte.

Die von Neumann-Architektur vereinheitlicht Daten- und Programmspeicher.

EINSEITIG UNENDLICHES BAND

Beidseitig unendliches Band kann durch ein Alphabet doppelter Wortbreite realisiert werden, somit äquivalent.

TODO:
diagram (buch 235)

TURING-ERKENNBARE SPRACHEN

Eine TM erkennt das Wort $w \in \Sigma^*$, wenn die Maschine auf dem Input w im Zustand q_{accept} anhält.

Sind L_1 und L_2 Turing-erkennbare Sprachen, dann sind auch der Durchschnitt $L_1 \cap L_2$ und die Vereinigungsmenge $L_1 \cup L_2$ Turing-erkennbar.

Es gibt überabzählbare viele Sprachen $L \subset \Sigma^*$, die **nicht** Turing-erkennbar sind.

AUFZÄHLER

Aufzähler: TM mit einem Drucker, mit dem Wörter ausgedruckt werden können.

L ist **rekursiv aufzählbar** wenn es einen Aufzähler gibt, der alle Wörter aufzählen kann. L ist dann auch Turing-erkennbar.

ENTSCHEIDER UND ENTSCHEIDBARE SPRACHEN

Turing-erkennbare Sprache: L heisst **Turing-erkennbar**, wenn es eine TM M gibt mit $L = L(M)$.

Turing-entscheidbare Sprache: L heisst **Turing-entscheidbar**, wenn es einen Entscheider M gibt mit $L = L(M)$.

Berechenbare Funktion: Eine Funktion $f: \Sigma^* \rightarrow \Sigma^*$ heisst **berechenbar**, wenn es eine TM M gibt, die auf jedem Inputwort $w \in \Sigma^*$ anhält und auf dem Band das Outputwort $f(w)$ zurücklässt.

TODO:
nicht entscheidbare Probleme

ENTSCHEIDBARKEIT

«Die sich mit immer neuen Superlativen gegenseitig überbietenden Ankündigungen der IT-Industrie könnten den Eindruck erwecken, dass mit geeignet viel Rechenleistung und Speicherplatz kein Informatikproblem einer Lösung widerstehen könnte.»

— Prof. Dr. Andreas Müller, [1, p. 267]

> based

TODO:
(Zehntens hilbertsches Problem). Man gebe ein Verfahren an, welches für eine beliebige diophantische Gleichung entscheidet, ob sie lösbar ist.

Serialisierung ermöglicht, jedes Entscheidungsproblem auf die Verarbeitung einer Zeichenkette zurückzuführen, die entweder akzeptiert oder verworfen wird. Jedes Problem wird auf diese Weise zu einem Sprachproblem.

ENTSCHEIDER

Definition

Ein **Entscheider** ist eine Turing-Maschine, die auf jedem beliebigen Input anhält.

Beispiele

– Leerheitsproblem (Entscheidbar): $E_{\text{DEA}} = \{\langle A \rangle \mid A \text{ ein DEA und } L(A) = \emptyset\}$

– Gleichheitsproblem (Entscheidbar): $G = \{\langle G_1, G_2 \rangle \mid G_i \text{ CFGs und } L(G_1) = L(G_2)\}$

– Akzeptanzproblem (Entscheidbar – nicht für TM!): $A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ ist eine TM, die das Wort } w \in \Sigma^* \text{ akzeptiert}\}$

– Halteproblem (Nicht entscheidbar): $\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ hält auf Input } w\}$

AKZEPTANZPROBLEM FÜR TURING-MASCHINEN

$A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ ist eine TM, die das Wort } w \in \Sigma^* \text{ akzeptiert}\}$

Theorem

A_{TM} ist nicht entscheidbar.

Beweis

Konstruiere aus einem Entscheider H für A_{TM} eine Maschine D mit Input $\langle M \rangle$

1) Lasse H auf Input $\langle M, \langle M \rangle \rangle$

2) Falls H akzeptiert: q_{reject}

3) Falls H verwirft: q_{accept}

Wende jetzt D auf $\langle D \rangle$ an:

$D(\langle D \rangle)$ akzeptiert $\Rightarrow D$ verwirft $\langle D \rangle$

$D(\langle D \rangle)$ verwirft $\Rightarrow D$ akzeptiert $\langle D \rangle$

HALTEPROBLEM

$\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid \text{die TM } M \text{ hält auf dem Input } w\}$

$\Rightarrow$ Gleicher Widerspruch wie beim Akzeptanzproblem. Umschreiben des Problems in ein neues Programm S , welches «Akzeptieren» in «Anhalten» übersetzen soll = Reduktion ($A_{\text{TM}} \leq \text{HALT}_{\text{TM}}$)

TODO:
spezielles/allgemeines Halteproblem

REDUKTION

Reduktionsabbildung

Berechenbare Abbildung $f : \Sigma^* \rightarrow \Sigma^*$ so, dass

$w \in A \Leftrightarrow f(w) \in B$

Notation: $f : A \leq B$ heisst «A leichter entscheidbar als B»

Entscheidbarkeit

Falls B entscheidbar, dann $f : A \leq B \Rightarrow A$ entscheidbar

Beweis

Ist H ein Entscheider für B , dann ist $H \circ f$ ein Entscheider für A .

Folgerung

A nicht entscheidbar, $A \leq B \Rightarrow B$ nicht entscheidbar

VERGLEICH DER ENTSCHEIDBARKEIT VON SPRACHEN

Beispiel 18

TODO:

SATZ VON RICE

LÖSUNGSMETHODEN FÜR ENTSCHEIDUNGSPROBLEME

TODO:
book 273..

KOMPLEXITÄT
Ineffizient: Entscheidbare Probleme mit exorbitant langer Laufzeit sind praktisch «nicht lösbar»
Problemgrösse: Laufzeit hängt von der Problemgrösse n ab
Worst case Laufzeit: Die worst case Laufzeit $t(n)$, die die Maschine zur Verarbeitung von Inputs der Länge n braucht, ist $t(n) = \max\{t(w) \mid |w| \leq n\}$

HARDWAREEINFLUSS
Nichtdeterministische Hardware kann in Zeit $2^{O(t(n))}$ simuliert werden $\Rightarrow$ NTM ist exponentiell schneller

POLYNOMIELLE UND EXPONENTIELLE LAUFZEIT

TODO:
slides 5

Polynomielle Probleme	Exponentielle Probleme
- Gauss-Algorithmus $O(n^3)$ - ...	- Faktorisierung von grossen Zahlen (RSA) - ...
Skalierbarkeit	Sicherheit

Ein Algorithmus hat **polynomielle Laufzeit**, wenn seine Laufzeit durch ein Polynom beschränkt ist: $t(n) = O(n^k)$.

SPRACHEN
Eine Sprache L gehört zur Klasse NP , wenn L von einer nichtdeterministischen TM in **polynomieller Zeit entschieden** werden kann

POLYNOMIELLE VERIFIZIERER
Ein **polynomieller Verifizierer** für die Sprache L ist eine TM V , so dass es für jedes $w \in \Sigma^*$ ein Wort c (das Lösungszertifikat) gibt, für das gilt

$w \in L \Leftrightarrow V$ akzeptiert $\langle w, c \rangle$

Die Laufzeit von V ist polynomiell in $|w|$.

Eine Sprache ist genau dann in **NP**, wenn sie in polynomieller Zeit **verifiziert** werden kann.

TODO:
slides 14

POLYNOMIELLE REDUKTION

POLYNOMIELLER LAUFZEIT-VERGLEICH

Seien A und B entscheidbare Sprachen. Eine berechenbare Abbildung

$f : \Sigma^* \rightarrow \Sigma^* : w \mapsto f(w)$

mit

1) $w \in A \Leftrightarrow f(w) \in B$ (Reduktion)
2) $f(w)$ ist berechenbar in polynomieller Zeit in $|w|$

heisst **polynomielle Reduktion** $A \leq_p B$. Lies: A ist polynomiell leichter entscheidbar als B .

Polynomiell äquivalent: Zwei Sprachen A und B heissen **polynomiell äquivalent**, geschrieben $A \equiv_p B$, wenn $A \leq_p B$ und $B \leq_p A$.

SATZ

Sei $A \leq_p B$. Dann gilt

$B \in P \Rightarrow A \in P, \quad A \notin P \Rightarrow B \notin P$

$B \in NP \Rightarrow A \in NP, \quad A \notin NP \Rightarrow B \notin NP$

TODO:
slides 16,17

REDUKTION VON HAMPATH

TODO:

POLYNOMIELLE AUSFÜLLRÄTSEL
Ein polynomielles Ausfüllrätsel ist eine $n \times m$ -Tabelle, in die Zeichen eines Alphabets Σ eingefüllt werden müssen, so dass als logische Formel ausdrückbare Regeln eingehalten werden, die in polynomieller Zeit ausgewertet werden können. (Beispiel: Sudoku)

SAT: Boolean satisfiability problem: Gegeben einer aussagenlogischen Formel, gibt es eine Zusammensetzung der Variablenwerte, die die Aussage «Wahr» werden lassen?

$SAT = \{\varphi \mid \varphi \text{ ist eine erfüllbare logische Formel}\}$

Satz
Jedes polynomielle Ausfüllrätsel A lässt sich polynomiell auf SAT reduzieren.

Variablen
 $x_{ijc} = \text{wahr}$
 $\Leftrightarrow$ Feld (i, j) der Tabelle enthält Zeichen c

Genau ein Zeichen im Feld (i, j)

TODO:

$$\varphi_{ij} = \bigvee_{c \in \Sigma} \left(\underbrace{x_{ijc} \wedge \bigvee_{d \neq c} x_{ijd}}_{c \text{ und kein anderes Zeichen in } (i, j)} \right)$$

Regeln
Spielregeln als Formel φ in Variablen x_{ijc} ausdrücken, die wahr ist genau dann, wenn die Regeln erfüllt sind.

NP-VOLLSTÄNDIGKEIT

Eine entscheidbare Sprache B heisst **NP-vollständig**, wenn sich jede Sprache A in NP polynomiell auf B reduzieren lässt:

$A \leq_p B \quad \forall A \in NP$

leichter schwieriger

NP-vollständige Probleme sind die «schwierigsten» Probleme in NP. Sie sind alle gleich schwierig:

A, B NP-vollständig $\Rightarrow (A \leq_p B \wedge B \leq_p A \Leftrightarrow A \equiv_p B)$

$BIP = \{\langle C, d \rangle \mid C \in M_{n \times m}(\mathbb{Z}), d \in \mathbb{Z}^n, \text{ gibt es } x \in \{0, 1\}^m \text{ sodass } Cx = d\}$

$CLIQUE-COVER = \{\langle G, k \rangle \mid\}$

Reduktionsprinzip
 A NP-vollständig $\wedge B \in NP \wedge A \leq_p B \Rightarrow B$ NP-vollständig

TODO:
slides 2

Wenn ein Problem NP-vollständig ist:
– Lösung braucht typischerweise exponentielle Zeit
– Korrektheit der Lösung ist in polynomieller Zeit zu prüfen

Umgang mit NP-Vollständigkeit in der Praxis:
– Spezialfälle ausnützen
– Problem modifizieren
– Approximation
– Zufall oder Heuristik
– Genetische Algorithmen und ML

Beispiele:
– SAT / 3SAT
– K-VERTEX-COLORING

COOK-LEVIN

3-SAT: Jede Klausel der Normalform hat maximal 3 Literale.

Satz
 SAT ist NP-vollständig.
Das bedeutet:
Sei A eine Sprache in NP
 $\Rightarrow$ Es gibt eine nichtdeterministische TM M , die A in polynomieller Zeit $O(t(n))$ entscheidet
 $\Rightarrow A$ kann polynomiell auf SAT reduziert werden: $A \leq_p SAT$

Vorgehensweise
Beschreibe das Finden der Berechnungsgeschichte von M als ein polynomielles Ausfüllrätsel

TODO:
slides 5

TODO:
slides 7

«Ausfüllrätsel» in diesem Fall die Berechnungsgeschichte der TM.
Überprüfe, ob alle Zustandsübergänge valid sind.

TODO:
book 328-343/368

SAT $\leq_p$ 3SAT

TODO:
 $\rightarrow$ Erfüllungsäquivalenz

Beweis von Cook-Levin
– Jedes Problem lässt sich auf eine Formel reduzieren
– Die Reduktion ist polynomiell
– Die Formel wird aus der Berechnungsgeschichte abgeleitet
– Die Formel kann in CNF konstruiert werden

Erfüllungsäquivalente Umformung

TODO:
slides 11

ÖFFENTLICHES SCHLÜSSELSYSTEM

TODO:
slides 15

KARP-KATALOG

TODO:
slides 16